ReproBreak: A Dataset of Reproducible Web Locator Breaks

Thiago Santos de Moura
Ruhr-Universität Bochum
Bochum, Germany
thiago.santosdemoura@rub.de

Leon Adamietz
Ruhr-Universität Bochum
Bochum, Germany
leon.adamietz@rub.de

Samra Mehboob
Ruhr-Universität Bochum
Bochum, Germany
samra.mehboob@rub.de

Yannic Noller
Ruhr-Universität Bochum
Bochum, Germany
yannic.noller@acm.org

Abstract

Automated GUI testing frameworks such as Cypress and Playwright rely on locators to find and interact with web elements. A locator break occurs when a structural change in the application under test causes a locator to no longer find its target element, resulting in test breakages even when the underlying functionality remains unchanged. Despite its impact on test maintenance, no dataset exists to evaluate locator fragility in Cypress and Playwright at scale. In this paper, we present ReproBreak, a dataset of reproducible locator breaks in web application GUI tests. We analyzed 359 open-source repositories to identify commits that contain locator changes. To confirm whether these changes are indeed locator breaks, we reproduced them in the top 4 projects with the largest number of locator changes and found 449 locator breaks, which are provided in the dataset along with scripts for automated reproduction. We believe ReproBreak serves as a valuable artifact to support research on locator fragility, repair techniques, and test robustness. The video is available at: https://youtu.be/mZByS_TnCvE. The dataset is at https://github.com/rub-sq/ReproBreak.

ACM Reference Format:
Thiago Santos de Moura, Leon Adamietz, Samra Mehboob, and Yannic Noller. 2026. ReproBreak: A Dataset of Reproducible Web Locator Breaks. In . ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern web applications are present across most software systems, ranging from e-commerce platforms to public services. As these applications evolve rapidly, the risk of functional and visual regressions rises, making reliable Graphical User Interface (GUI) testing increasingly important. Frameworks such as Cypress [2] and Playwright [16] are widely adopted in the industry to automatically simulate user interactions, verify GUI behaviors, and detect regressions [5]. Unlike unit tests, GUI tests validate the system as a whole and detect integration gaps in a real environment, which is especially relevant for complex web applications.

Although these frameworks are effective, the tests written and executed with them rely on element locators, e.g., IDs, CSS selectors, or XPath expressions, to identify and interact with GUI elements. When the Application Under Test (AUT) changes, these locators may no longer point to the intended elements, causing the test to break. Those changes can be categorized into two families: *logical* and *structural* [13]. A logical change modifies the AUT's behavior to introduce new features or alter existing ones, requiring updates to the test cases, but typically not to the locators themselves. A structural change, on the other hand, affects the layout or organization of the GUI without necessarily changing the underlying functionality, for example, renaming an element ID or restructuring the DOM [3]. Such changes frequently invalidate existing locators, causing tests to break even when the application behavior remains correct. This phenomenon is known as a locator fragility or breakage [22].

In this work, a modification to a test locator following the evolution of the AUT is called a locator change (L_c). A locator break (L_b) occurs when the previous locator can no longer identify the intended element in the updated application. Not every locator change is a break. A non-breaking locator change (L_{nb}) occurs when the original locator would still work, meaning the update was merely prophylactic. Every locator change thus belongs to one of these two categories, i.e., $L_c = L_b \cup L_{nb}$. Figure 1 illustrates such a scenario, where a locator used by Test v1 to identify a button was renamed in App v2, causing a breakage when Test v1 is executed on App v2. To determine which case applies, given a commit that modifies a locator, we first verify that the new locator passes on the updated application, and then reinsert the old locator into the same commit. If the test fails, we classify it as a L_b. If it passes, the change was not driven by a break (L_{nb}).

```
// Test v1
cy.get('.btn-action')
    .should('be.visible');

// App v1
<button class="btn-action">
  Action
</button>
```

Test v1 / App v1

```
// Test v2
cy.get('.action-btn')
    .should('be.visible');

// App v2
<button class="action-btn">
  Action
</button>
```

Test v2 / App v2

Figure 1: Locator break example. Renaming the CSS class in App v2 invalidates the locator in Test v1.

Locator fragility has been extensively documented in the literature. Christophe et al. [1] found that conventional locators caused up to 75% of Selenium test files to change every nine commits. An analysis of 1,065 test breakages across 453 web application versions

Conference'17, Washington, DC, USA

showed that fragile locators caused 73.6% of the failures [8]. A systematic literature review [18] further identified robust identification of web elements as the most prominent challenge in GUI test automation, and a recent study [20] pointed out that the creation of test scripts that are overly sensitive to small application changes is one of the three main problems in GUI testing.

Several techniques have been proposed to address this problem, including more resilient locator generation [14], similarity-based repair [17, 19], and NL-based testing approaches [11]. However, these were primarily developed and evaluated for Selenium, which sends commands to the browser through an *external* driver. Cypress, on the other hand, runs directly *inside* the browser alongside the application, while Playwright controls patched versions of the browser through *internal* APIs [6]. This means the way these frameworks query and interact with elements differs considerably, and no dataset exists to evaluate and compare locator fragility techniques for Cypress and Playwright at scale. To fill this gap, we introduce **ReproBreak**, a dataset of reproducible locator breaks from open-source web applications using Cypress and Playwright for GUI testing. In summary, this paper makes the following contributions:

(1) Analyzing 359 open-source repositories that use Playwright (189), Cypress (154), or both (16) for GUI testing to identify locator changes and breaks.
(2) Creating a structured dataset of locator changes and breaks.
(3) Providing scripts to automatically reproduce locator breaks, to facilitate further research and tool development.

2 Dataset Construction

2.1 Data Source

Our dataset is derived from the E2EGit dataset [15], which consists of 472 open-source web application projects, each containing a web GUI test suite implemented in one of the following frameworks: Cypress [2], Playwright [16], Puppeteer [7], or Selenium [21]. From this dataset, we extracted GitHub repository links and filtered the projects to those that use Cypress or Playwright, resulting in 374 open-source repositories, 191 for Playwright and 183 for Cypress.

2.2 Data Collection

Our data collection pipeline (see Figure 2) consists of three steps: identification of locator changes, Docker-based environment setup, and locator break validation. The collection process is automated using a Python script, except for the creation of the reproduction files used in the Docker-based environment setup, which were crafted manually. The cut-off date for the dataset is April 21, 2026.

Identification of Locator Changes. After cloning the 374 repositories, we identify all test files in the latest commit, following the same approach as E2EGit [15]. We filter by file extension to support only Java, JavaScript/TypeScript, and Python files. For each language, we apply rules that suggest test file formats, e.g., `.spec.ts` for TypeScript. When we find such a file, we read its content and apply a regex to detect framework-specific keywords, such as `cy.get` for Cypress or `page.locator` for Playwright, to confirm it contains web GUI tests. Next, we parse the output of `git log` to identify which commits modified each test file, mapping each test file path to the commits that changed it. For each commit where a test file was modified, we store the commit hash, date, and previous commit hash, so we can later inspect both versions. Finally, for each commit pair we run `git diff` to compute the differences, resulting in a list of pairs of code hunks that pinpoint the changes. Each hunk contains the old and new versions of a changed section. We apply regex patterns to detect locator usages in both versions. These patterns were designed to match specific method calls, such as `cy.get` in Cypress or `page.locator` in Playwright, and extract their parameters. When we find a locator removal and the addition of another in consecutive lines within a hunk, we classify it as a locator change (L_c).

Repository	$\#L_c$	$\#L_c^{valid}$	$\#L_b$	$\#L_{nb}$
ghiscoding/angular-slickgrid	344	270	258	12
nasa/openmct	271	88	48	40
tryghost/koenig	264	155	93	62
microsoft/playwright	399	67	50	17
Total	1,278	580	449	131

Table 1: Number of locator changes ($\#L_c$), inspected ($\#L_c^{valid}$), locator breaks ($\#L_b$), and non-breaking ($\#L_{nb}$) per repository. $\#L_c^{valid}$ corresponds to commits that are validated.

Docker Environment Setup. To reproduce a possible locator break, we need to run both the application and the test file. Since each project has different dependencies, such as a local database like PostgreSQL or whole programming languages like PHP, we encapsulate everything in a Docker-based environment to ensure consistent and reproducible execution across machines. Given the manual effort to set up these environments, we focused on the top-10 projects with the highest number of locator changes. From those, we were able to create reproduction files only for four, shown in Table 1. The remaining six could not be reproduced due to insufficient documentation, inaccessible dependencies, or unavailable environment configurations. For each project, the reproduction files consist of two components: a Dockerfile that sets up the application and all its dependencies, and a script that starts the application and executes a given test file. These files are first created for the latest version of the application (step 2a). The Docker image is built once and reused across all test files in that commit, avoiding redundant rebuilds. In step 2b, we evaluate the reproduction files against all identified commits. For commits where the build or execution fails, we refine the reproduction files and repeat the evaluation (step 2c) until no further adjustments can be made. We store the reproduction file tuples and their commits in the database.

Locator Break Validation. With the locator changes and reproduction files in place, we validate each inspected L_c (steps 3a–3c in Figure 2). In step 3a, we execute the test file using the new locator on the updated application. If the tests pass, we replace the new locator with the old one in step 3b and execute the tests again in step 3c. If the test now fails, we classify it as a locator break (L_b) because the only difference between the two executions is the locator itself. If it passes, we classify it as a non-breaking locator change (L_{nb}).

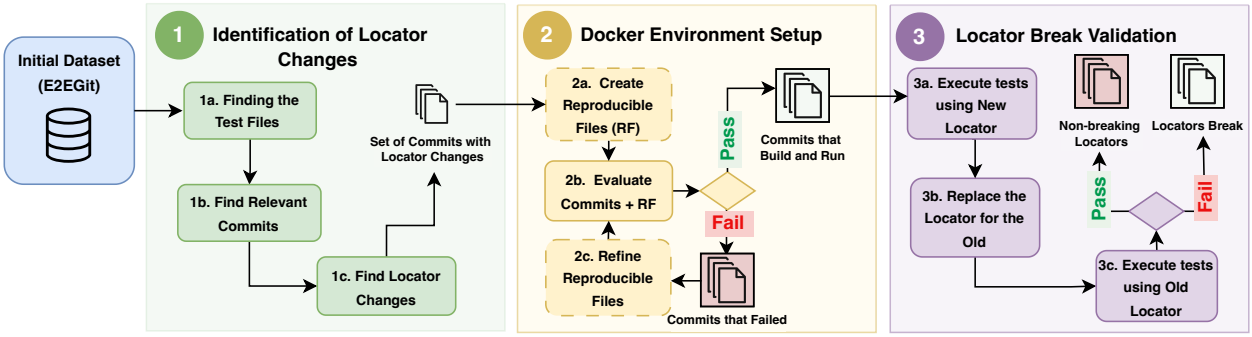

Figure 2: Data Collection Pipeline. Dashed boxes (step 2) indicate manually efforts.

Framework	L_c	Projects with L_c
Playwright	4,186	111/189 (58.7%)
Cypress	4,026	86/154 (55.8%)
Both Frameworks	1,360	14/16 (87.5%)
Overall	9,572	211/359 (58.7%)

Table 2: Summary of locator changes found per framework.

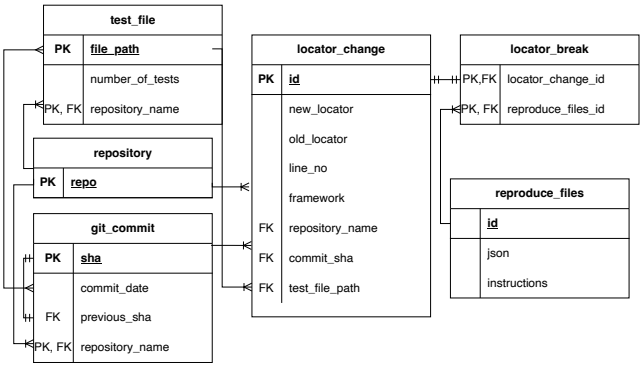

Figure 3: Entity-relationship model of the dataset.

Command	Mode `m`	Expected Result
`uv run reproduce.py -locator_id <id> -mode <m>`	fixed	✓ Test passed
	reproduce_break	× Test failed
	overwrite	? Depends on changes

Table 3: Reproduction script execution modes.

3 Dataset Description

3.1 Dataset Overview

The dataset is stored as a relational SQLite database (see schema in Figure 3). The schema is centered around the `locator_change` table, which stores the old and new locator, the line number, the framework, and references to the repository, commit, and test file where the change occurred. The `git_commit` table captures the version history, storing each commit's hash, date, and a reference to its previous commit, so that any locator change can be traced back to both versions of the code. The `locator_break` table connects a locator change to the reproduction files used to validate it, and the `reproduce_files` table stores the Dockerfiles and shell scripts as JSON. The `repository` and `test_file` tables store project-level and file-level metadata to support filtering and analysis. Table 2 shows the number of locator changes identified from 211 out of the 359 repositories. Overall, we identified 9,572 locator changes. The dataset, along with all scripts used to collect it and reproduce locator breaks, is publicly available in our artifact.

3.2 Reproduce a Locator Break

The `reproduce.py` script orchestrates the full workflow: retrieves files from the database, sets up the test environment, and executes the tests. It takes a locator break *ID* and one of three execution *modes* as arguments (see Table 3). In `fixed` mode, the script executes the test file using the new locator on the updated application, and the test passes, confirming the locator change was valid. In `reproduce_break` mode, it reinserts the old locator into the same commit and reruns the tests, which fail, confirming the locator break. The `overwrite` mode checks whether a custom repair resolves the break. During the first execution, the Docker image for the specified commit is built automatically (see Figure 4). Subsequent runs reuse the cached image. At the end of each execution, the script outputs the test file path, allowing researchers to inspect the test, modify the locator, and rerun the validation.

4 Application Scenarios

ReproBreak's reproducible locator breaks enable several empirical studies and tool development efforts in web GUI test maintenance. In this section, we present four application scenarios of ReproBreak.

Locator Robustness Techniques. Approaches such as Robula+ [14] and Sidereal [12] derive more resilient locators, but have been built and evaluated exclusively for Selenium. Given that Cypress and Playwright rely on different element identification mechanisms, it is still unknown whether these techniques work also for Cypress and Playwright. Now, ReproBreak enables such evaluation.

Fragility Score Assessment. The fragility score [4], originally proposed and evaluated for Selenium, needs to be adapted for Cypress

```
uv run reproduce.py --locator_id 1224 --mode
    reproduce_break
Getting info for locator break ID: 1224
  Repository: ghiscoding/angular-slickgrid
  Commit: b779ef14186481ebbd3ff32c1167729989feb3cf
Cloning repo: ghiscoding/angular-slickgrid
Extracting reproduce files from database...
  / Dockerfile
  / run_tests.sh
Setting up Docker image...
Docker image already exists. Skipping build.
Replacing locator to reproduce break...
Running GUI-tests tests...
Locator information
File path: /Users/../ReproBreak/data/...
Line number: 440
Locator change: cy.get('.slick-header-menu') ->
    cy.get('.slick-header-menu .slick-menu-command-list')
Test failed!
```

Figure 4: Example of reproduce.py script execution.

and Playwright, given the distinct locator strategies these frameworks use. Once adapted, ReproBreak can be used to evaluate it. Furthermore, analyzing which locator types present fewer breaks across the dataset can provide empirical grounding to refine the score and better capture what makes a locator break-prone.

Locator Repair Approaches. Our dataset can be used to assess and compare locator repair approaches. Techniques originally developed for Selenium, such as Similo [17], Vista [22], and Color [10], can now be evaluated on Cypress and Playwright projects using ReproBreak. Additionally, LLM-based agents for automated locator repair could be developed and benchmarked using our dataset.

NL-Based Testing Robustness. ReproBreak can also be used to evaluate the robustness of NL-based testing techniques [11] regarding locator breaks. A potential direction could be to convert the test that uses the old locator into an NL-based representation and assess whether the abstraction level could have prevented the break. This was initially investigated on a smaller scale using Selenium [9], and ReproBreak enables this evaluation at a larger scale.

5 Conclusion

We presented ReproBreak, a dataset of reproducible locator breaks from open-source web applications using Cypress and Playwright. To the best of our knowledge, this is the first dataset of its kind for these frameworks. We analyzed 359 repositories, identified 9,572 locator changes, and reported 449 reproducible locator breaks across four projects. The scripts for automated reproduction make ReproBreak particularly valuable. Rather than just providing static snapshots of broken locators, the dataset allows researchers to actively confirm breaks, test repair techniques, and benchmark their approaches in a real execution environment. Other researchers can easily extend ReproBreak to other frameworks by simply defining new locator regex patterns. In the future, we will extend ReproBreak to more projects and explore using LLM-based agents to reduce the manual effort in the reproduction.

References

[1] Laurent Christophe, Reinout Stevens, Coen De Roover, and Wolfgang De Meuter. 2014. Prevalence and Maintenance of Automated Functional Tests for Web Applications. In *International Conference on Software Maintenance and Evolution (ICSME '14)*. IEEE CS, USA, 141–150. doi:10.1109/ICSME.2014.36

[2] Cypress.io. 2026. Cypress: Modern Web Testing Framework. https://www.cypress.io/. Accessed: 2026-05-06.

[3] Marco De Luca, Anna Rita Fasolino, and Porfirio Tramontana. 2024. Investigating the robustness of locators in template-based Web application testing using a GUI change classification model. *JSS* 210 (April 2024), 16 pages. doi:10.1016/j.jss.2023.111932

[4] Sergio Di Meglio and Luigi Libero Lucio Starace. 2024. Towards Predicting Fragility in End-to-End Web Tests. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy) *(EASE '24)*. ACM, New York, NY, USA, 387–392. doi:10.1145/3661167.3661179

[5] Sergio Di Meglio, Luigi Libero Lucio Starace, Valeria Pontillo, Ruben Opdebeeck, Coen De Roover, and Sergio Di Martino. 2026. Investigating the adoption and maintenance of web GUI testing: Insights from GitHub repositories. *IST* 189 (2026), 107928. doi:10.1016/j.infsof.2025.107928

[6] Boni García, Jose M. del Alamo, Maurizio Leotta, and Filippo Ricca. 2024. Exploring Browser Automation: A Comparative Study of Selenium, Cypress, Puppeteer, and Playwright. In *QUATIC*. Springer Nature Switzerland, Cham, 142–149.

[7] Google. 2026. Puppeteer: Headless Chrome Node.js API. https://pptr.dev/. Accessed: 2026-05-06.

[8] Mouna Hammoudi, Gregg Rothermel, and Paolo Tonella. 2016. Why do Record/Replay Tests of Web Applications Break?. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 180–190. doi:10.1109/ICST.2016.16

[9] Hiroyuki Kirinuki, Shinsuke Matsumoto, Yoshiki Higo, and Shinji Kusumoto. 2022. Web Element Identification by Combining NLP and Heuristic Search for Web Testing. In *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 1055–1065. doi:10.1109/SANER53432.2022.00123

[10] Hiroyuki Kirinuki, Haruto Tanno, and Katsuyuki Natsukawa. 2019. COLOR: Correct Locator Recommender for Broken Test Scripts using Various Clues in Web Application. In *IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 310–320. doi:10.1109/SANER.2019.8667976

[11] Maurizio Leotta, Filippo Ricca, Alessandro Marchetto, and Dario Olianas. 2024. An empirical study to compare three web test automation approaches: NLP-based, programmable, and capture&replay. *Journal of Software: Evolution and Process* 36, 5 (2024), e2606. doi:10.1002/smr.2606

[12] Maurizio Leotta, Filippo Ricca, and Paolo Tonella. 2021. Sidereal: Statistical adaptive generation of robust locators for web testing. *Software Testing, Verification and Reliability* 31, 3 (2021), e1767. doi:10.1002/stvr.1767

[13] Maurizio Leotta, Andrea Stocco, Filippo Ricca, and Paolo Tonella. 2015. Using Multi-Locators to Increase the Robustness of Web Test Cases. In *IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. 1–10. doi:10.1109/ICST.2015.7102611

[14] Maurizio Leotta, Andrea Stocco, Filippo Ricca, and Paolo Tonella. 2016. Robula+: an algorithm for generating robust XPath locators for web testing. *Journal of Software: Evolution and Process* 28, 3 (2016), 177–204. doi:10.1002/smr.1771

[15] Sergio Di Meglio, Luigi Libero Lucio Starace, Valeria Pontillo, Ruben Opdebeeck, Coen De Roover, and Sergio Di Martino. 2025. E2EGit: A Dataset of End-to-End Web Tests in Open Source Projects. In *IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 836–840. doi:10.1109/MSR66628.2025.00121

[16] Microsoft. 2026. Playwright: Fast and reliable end-to-end testing for modern web apps. https://playwright.dev/. Accessed: 2026-05-06.

[17] Michel Nass, Emil Alégroth, Robert Feldt, Maurizio Leotta, and Filippo Ricca. 2023. Similarity-based Web Element Localization for Robust Test Automation. *ACM TOSEM* 32, 3, Article 75 (April 2023), 30 pages. doi:10.1145/3571855

[18] Michel Nass, Emil Alégroth, and Robert Feldt. 2021. Why many challenges with GUI test automation (will) remain. *IST* 138 (2021), 106625. doi:10.1016/j.infsof.2021.106625

[19] Michel Nass, Emil Alégroth, Robert Feldt, and Riccardo Coppola. 2023. Robust web element identification for evolving applications by considering visual overlaps. In *IEEE Conference on Software Testing, Verification and Validation (ICST)*. 258–268. doi:10.1109/ICST57152.2023.00032

[20] Filippo Ricca, Maurizio Leotta, and Andrea Stocco. 2019. Three Open Problems in the Context of E2E Web Testing and a Vision: NEONATE. Advances in Computers, Vol. 113. Elsevier, 89–133. doi:10.1016/bs.adcom.2018.10.005

[21] Selenium Project. 2026. Selenium: Browser Automation Framework. https://www.selenium.dev/. Accessed: 2026-05-06.

[22] Andrea Stocco, Rahulkrishna Yandrapally, and Ali Mesbah. 2018. Visual web test repair. In *26th ACM ESEC/FSE*. ACM, New York, NY, USA, 503–514. doi:10.1145/3236024.3236063